

SkillPath

An AI-aware developer job role and career path recommendation system

Technical Report — Stage 6, Stage 7 and Stage 8

Model Development · Model Optimisation and Final Model Selection · Evaluation and Defence

Group member	Registration number
S S P S Bandara	IT23602250
Yoosuf A.A	IT23645202
M G S D Wijesinghe	IT23564640
K M H S Bandara	IT23792418

Headline result on the held-out test split (4,615 respondents)

83.9% correct job role in the top 3 shown	87.8% correct career family in the top 3	0.284 macro-F1 (20 classes)	54.9% top-1 accuracy
---	--	---	--

Final model: Logistic Regression (C = 0.218, no class weighting) inside the Stage 4 preprocessing pipeline. Group KND_12, SLIIT Kandy Uni.

ORIENTATION

Executive summary

This report covers Stage 6 (model development), Stage 7 (optimisation and final model selection) and the material prepared for Stage 8 (individual evaluation). Stages 3 and 4 are documented separately in *preprocessing_decisions.md*.

The task

Predict a developer's **specific job role** from a self-reported skill and AI-attitude profile: a 20-class classification problem over 479 features, built from 18,457 training and 4,615 held-out respondents. A role family's probability is the **sum** of its job roles' probabilities, so a single model produces both the three job-role suggestions and the career-family shortlist the application shows.

What was done

Stage	Work	Evidence
6	Seven candidates (one baseline, six algorithms) cross-validated and compared	stage6_model_comparison.csv
7A	Imbalance handling: none vs class weighting vs oversampling, on two algorithms	stage7_a_imbalance.csv
7B	Nine feature-set and feature-selection experiments	stage7_b_features.csv
7C	Randomised hyper-parameter search for all four serious models	stage7_c_tuned.csv
7D	Tuned vs baseline, final model chosen by a pre-registered rule	stage7_d_tuned_vs_baseline.csv
—	Final fit, single held-out evaluation, deployable artifact	artifacts/model.joblib

26 cross-validation runs are recorded with timestamps in *reports/model_experiments.csv*.

The three findings that mattered

- **Removing class weighting gained more than any hyper-parameter.** Stage 6 applied *class_weight="balanced"* reflexively because the data is imbalanced 155:1. Phase A showed it was *harming* Logistic Regression by 0.047 macro-F1 — about fifteen fold-standard-deviations, and larger than every tuning effect found later.
- **Optimisation changed which model ships.** Hist Gradient Boosting led Stage 6 (0.2724 macro-F1); after tuning, Logistic Regression leads (0.3030). Tuning only the Stage 6 winner would have locked in the wrong model.
- **The error analysis contradicted the EDA prediction.** Confusion was expected *within* families (Cloud Infrastructure and DevOps have cosine similarity 0.99). In fact all eight largest confusions cross family boundaries and point at Full-stack. The dominant problem is imbalance, not role overlap.

Model comparison at a glance

Model	Stage 6 macro-F1	Stage 7 tuned	Test macro-F1
Logistic Regression	0.2547	0.3030	0.2845
Hist Gradient Boosting	0.2724	0.2860	—
Random Forest	0.2537	0.2820	—
Linear SVM (calibrated)	0.1911	0.2536	—
Complement Naive Bayes	0.1786	not tuned	—
k-Nearest Neighbours	0.1676	not tuned	—

Model	Stage 6 macro-F1	Stage 7 tuned	Test macro-F1
Dummy (majority class)	0.0283	—	—

STAGE 6 — MODEL DEVELOPMENT

Problem, target and validation strategy

Two properties of the target drive every decision in Stages 6 and 7.

1. Severe class imbalance

Full-stack accounts for 7,279 of the 18,457 training respondents (39.4%); the smallest role, UX / Research Ops / UI design, has 47. That is a ratio of about **155:1**. Always answering "Full-stack" scores 39.4% accuracy while knowing nothing, so accuracy cannot be the headline metric.

2. Genuinely overlapping classes

EDA measured cosine similarity 0.99 between the technology profiles of Cloud Infrastructure and DevOps engineers, and 0.97 between Data Engineer and Data Scientist. These roles are not separable *in the data*, so no algorithm can separate them. This sets the ceiling, and it is why the product shows **three ranked roles plus the family total** rather than a single answer.

Validation strategy

StratifiedKFold(`n_splits=5`, `shuffle=True`, `random_state=42`) on the training split, for every experiment in both stages, so every number in this report is comparable with every other.

Choice	Reason	Alternative rejected
Stratified	With 47 examples in the smallest class, an unstratified fold could contain almost none of it	Plain KFold: rare roles vanish at random
5 folds	The smallest class still contributes about 9 respondents per fold	10 folds leaves ~4 (per-class F1 becomes noise); 3 folds wastes training data
shuffle, fixed seed	Rows arrive in survey order; a fixed seed makes runs reproducible	Unshuffled: survey ordering becomes a fold effect
Preprocessor inside the estimator	Each fold refits vocabulary, rare pooling, medians, scaler and variance filter on its own training part	Fitting once on all training data: every validation fold would influence its own features
Test split untouched	Opened once, by <code>finalise_model.py</code> , after the model was chosen	Re-scoring on test while iterating turns it into a second validation set

Leakage controls

- Deterministic cleaning runs before the split; nothing statistical is learned first.
- Train and test IDs are disjoint and frozen in `split_ids.csv`.
- Resampling lives inside the estimator, so duplicated respondents can never appear on both sides of a fold.
- Feature selection lives inside the pipeline, so features are chosen per fold, never using labels the validation fold is meant to test.
- 19 automated checks in `tests/test_modelling.py` enforce these.

STAGE 6 — MODEL DEVELOPMENT

Algorithms and evaluation metrics

Stage 6 requires at least four algorithms; **six** are compared, plus a baseline. They were chosen to span different inductive biases rather than to be six variations of one idea.

One product constraint eliminated otherwise valid candidates

The application ranks three roles and sums their probabilities into families, so every candidate must produce usable *predict_proba*. A model without probabilities cannot be deployed whatever it scores. This is why LinearSVC appears wrapped in CalibratedClassifierCV rather than on its own — a modelling decision driven by the product, not by the metric.

Model	Inductive bias	Why it is in the comparison
Dummy (most frequent)	none	Fixes the floor and proves the metric choice: 0.394 accuracy with 0.028 macro-F1.
Complement Naive Bayes	features independent given the class	Built for imbalanced sparse count data. Its assumption is known false here (React implies JavaScript), so it measures what that assumption costs.
k-Nearest Neighbours	local similarity	Tests the product's own premise — similar skills, similar job. Cosine distance because the vectors are sparse 0/1 sets of very different breadth.
Logistic Regression	linear in feature space	The standard strong baseline for wide sparse data. Readable coefficients matter for explaining the system to non-technical stakeholders.
Linear SVM (calibrated)	maximum margin	Optimises margins rather than likelihood. Wrapped in Platt scaling because a bare SVM has no probabilities.
Random Forest	axis-aligned splits, bagged	Represents skill combinations a linear model cannot (Terraform AND Kubernetes AND Python implies AI/ML engineer).
Hist Gradient Boosting	additive boosted trees	Each round corrects the previous rounds' errors, which usually helps rare roles. Costly: 20 classes means 20 trees per round.

Evaluation metrics

Macro-F1 is primary because the product's value lies in the rare roles: a system that only ever recognises Full-stack and Back-end developers has nothing to tell the users who most need career guidance.

Metric	Level	Role in the report
macro-F1	job role	Primary. Equal weight per role, so neglecting UX/UI costs as much as neglecting Full-stack
balanced accuracy	job role	Mean per-class recall — is each role found at all
top-3 accuracy	job role	What the user actually sees
log loss	job role	Are the probabilities trustworthy? The app prints them as numbers
family macro-F1	family	The coarser recommendation, from the same summed probabilities
family top-3	family	The career-path shortlist
accuracy	job role	Reported only to demonstrate that it misleads

STAGE 6 — RESULTS

Model comparison

Five-fold cross-validation on the 18,457 training respondents. The test split was not opened at this stage.

Model	macro-F1	SD	bal. acc	accuracy	top-3	log loss	family F1	fit (s)
Hist Gradient Boosting	0.2724	0.0095	0.2477	0.5602	0.8289	1.474	0.3926	179.5
Logistic Regression	0.2547	0.0031	0.3321	0.3657	0.7262	2.269	0.3485	91.7
Random Forest	0.2537	0.0082	0.2404	0.5541	0.8301	1.608	0.3750	4.3
Linear SVM (calibrated)	0.1911	0.0080	0.1776	0.5368	0.8092	1.535	0.3274	11.6
Complement Naive Bayes	0.1786	0.0024	0.2030	0.4917	0.7210	2.328	0.1583	2.5
k-Nearest Neighbours	0.1676	0.0038	0.1510	0.4997	0.7814	4.413	0.2822	1.4
Dummy (most frequent)	0.0283	0.0000	0.0500	0.3944	0.4117	21.829	0.0471	1.8

Source: reports/stage6_model_comparison.csv. The highlighted row is the model that eventually shipped, after Stage 7 optimisation.

The Dummy row is the argument for the metric choice

The majority-class baseline scores **0.394 accuracy** — higher than Complement Naive Bayes (0.492 accuracy but far better macro-F1) would suggest, and higher than many readers expect of a model that cannot do anything at all. Its macro-F1 of 0.028 exposes it instantly. A report that led with accuracy would rank a useless model above models that have genuinely learned something.

Why the models rank as they do

- **Complement Naive Bayes (0.179)**. Fast and well above chance, but its independence assumption is false here: React implies JavaScript, Kubernetes implies Docker. It counts each co-occurring technology as fresh evidence and becomes over-confident — visible in its poor log loss (2.33) and very weak family F1 (0.158).
- **k-Nearest Neighbours (0.168)**. The product's premise holds weakly: top-3 is a respectable 0.781. But 479 dimensions is a hostile neighbourhood — with mostly 0/1 features, pairwise distances concentrate and the neighbourhood stops being local. Its log loss of 4.41 is the worst of any real model: the probabilities are nearly meaningless even where the ranking is not.
- **Logistic Regression (0.255) has the best balanced accuracy (0.332) and the worst accuracy (0.366)** of the serious models. That is *class_weight="balanced"* doing what it was asked: chase rare roles and accept false positives on Full-stack. Phase A revisits this with a surprising result.
- **Random Forest (0.254) and Hist Gradient Boosting (0.272)** represent skill combinations no linear model can. Boosting takes the top macro-F1 because each round focuses on the previous rounds' errors — which is where the rare roles live.

The headline finding is the spread, not the winner

The top three models sit at 0.2537, 0.2547 and 0.2724 with fold standard deviations of 0.008, 0.003 and 0.010. Random Forest and Logistic Regression differ by 0.001 — a tenth of a standard deviation, which is noise. Meanwhile the cost range is a factor of 40: Random Forest fits in 4.3 s, boosting in 179.5 s, for 0.019 macro-F1. The ceiling is set by the data, not by the algorithm.

STAGE 6 — RESULTS

Comparison charts

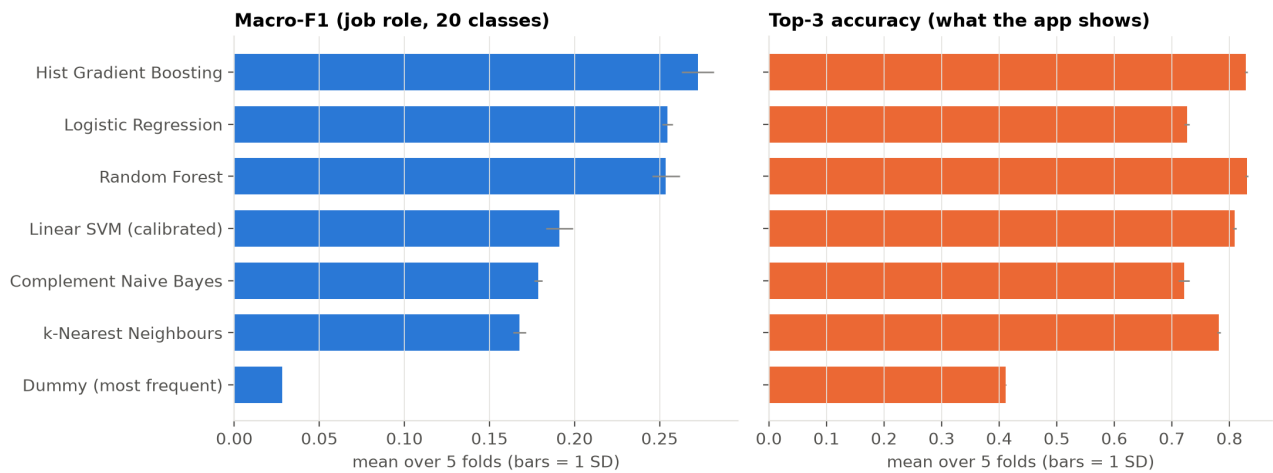


Figure 1. Macro-F1 and top-3 accuracy per model, with one standard deviation across the five folds. Note how differently the two panels rank the models: Logistic Regression is near the top on macro-F1 and near the bottom on top-3.

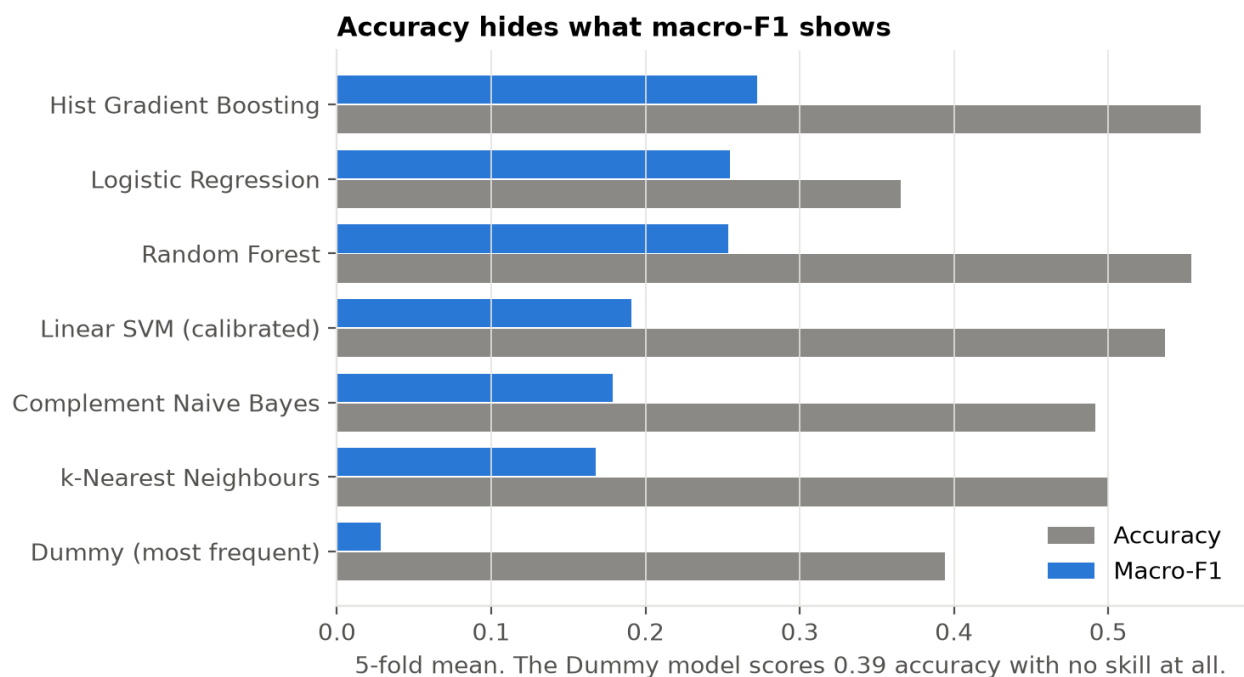


Figure 2. Accuracy against macro-F1. The Dummy model reaches 0.394 accuracy with no skill whatsoever, which is why accuracy is reported only as a cautionary number and macro-F1 is the primary metric.

STAGE 6 — ERROR ANALYSIS

Where the model fails, and why

Out-of-fold predictions from the best Stage 6 model, so no prediction comes from a fold that saw that respondent.

The expected error pattern was wrong, and it reordered the Stage 7 priorities

EDA predicted the damage would fall *within* families — Cloud Infrastructure confused with DevOps, Data Engineer with Data Scientist. Instead, **all eight largest confusions cross family boundaries and seven point at Full-stack**. These are not neighbouring roles with similar skills: Full-stack is 39% of the training data and behaves as a *sink* that absorbs anything the model is unsure about. The dominant failure is class imbalance, not role overlap — which is why Phase A became the first optimisation experiment rather than a formality.

Largest confusions

Actual role	Predicted as	Share	Same family?
UX / UI Designer	Full-stack Developer	0.60	no
Front-end Developer	Full-stack Developer	0.56	no
AI Application Developer	Full-stack Developer	0.50	no
System Administrator	Full-stack Developer	0.48	no
Cybersecurity Specialist	Full-stack Developer	0.42	no
QA / Test Engineer	Full-stack Developer	0.40	no
Desktop / Enterprise Developer	Full-stack Developer	0.39	no
Cloud Infrastructure Engineer	Back-end Developer	0.39	no

Per-class performance, best and worst five

Job role	precision	recall	F1	training examples
Full-stack Developer	0.610	0.839	0.706	7,279
Mobile Developer	0.713	0.696	0.704	739
Embedded Developer	0.504	0.575	0.537	771
Back-end Developer	0.512	0.515	0.514	3,750
Front-end Developer	0.635	0.380	0.476	1,120
... 10 roles omitted ...				
Cloud Infrastructure Engineer	0.189	0.042	0.069	238
QA / Test Engineer	0.238	0.025	0.045	199
Cybersecurity Specialist	0.172	0.023	0.040	220
AI Application Developer	0.000	0.000	0.000	116
UX / UI Designer	0.000	0.000	0.000	47

Two roles score exactly zero: the model never predicts them correctly out-of-fold. Mobile is the instructive exception — 739 examples but F1 0.704, as high as Full-stack's, because mobile developers use a distinctive toolset (Swift, Kotlin, Flutter) that nothing else uses. Where skills are distinctive a few hundred examples is plenty; where they overlap with Full-stack, even a thousand is not.

STAGE 6 — ERROR ANALYSIS

Diagnostic charts

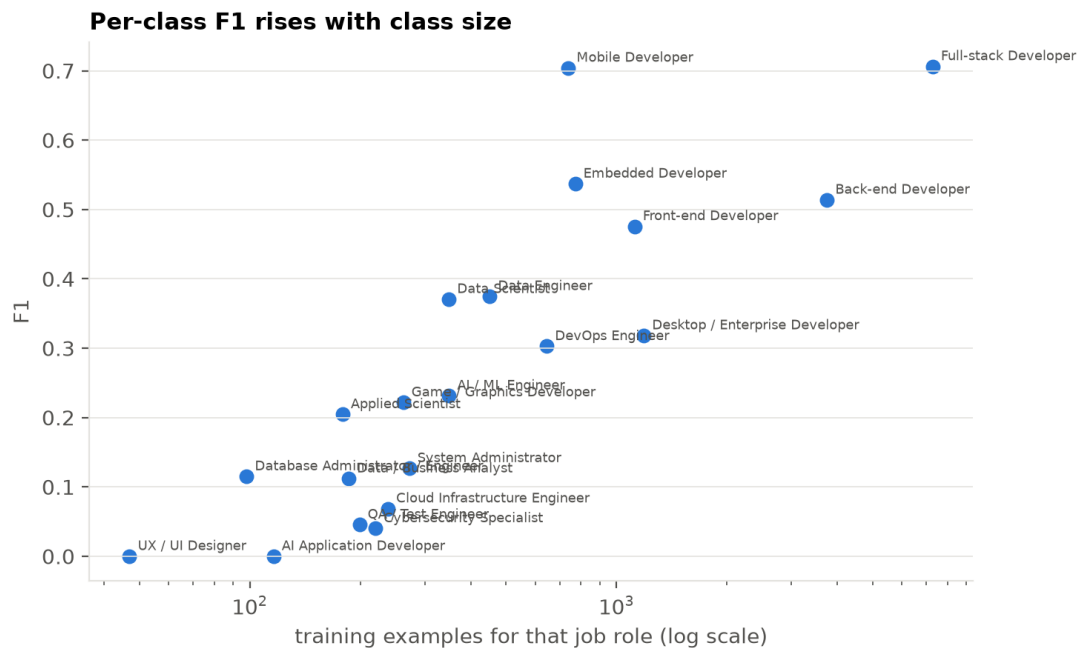


Figure 3. Per-class F1 against the number of training examples (log scale). The relationship is almost monotonic, which identifies class imbalance as the primary target for Stage 7. Mobile sits well above the trend because its toolset is distinctive.

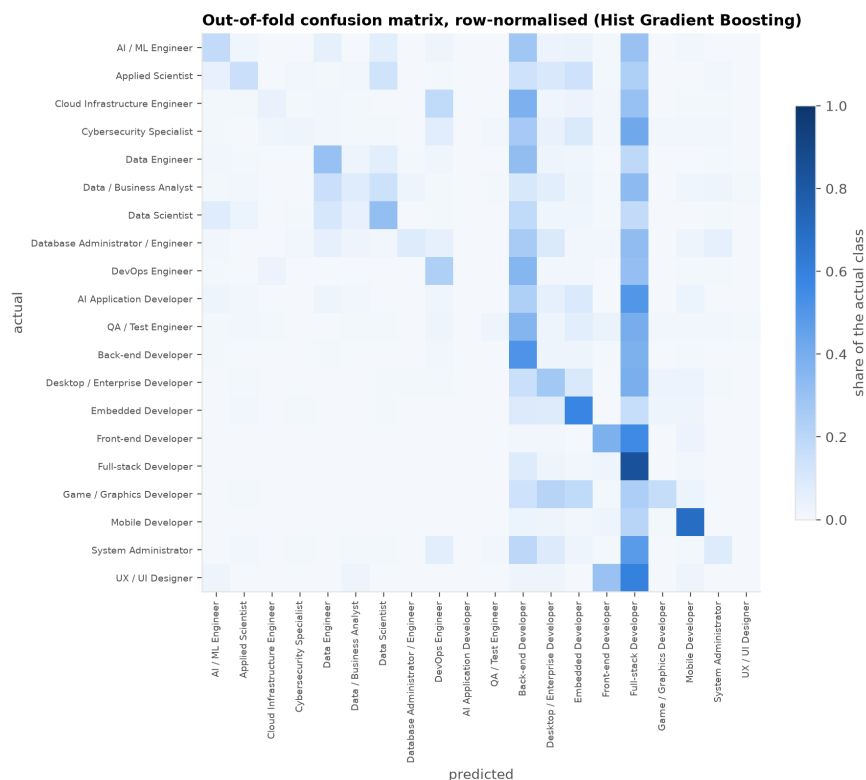


Figure 4. Row-normalised out-of-fold confusion matrix. The bright vertical band at Full-stack is the sink effect: most roles leak probability mass into the majority class.

STAGE 7 — PHASE A

Imbalance handling

Stage 6 showed per-class F1 tracking class size almost monotonically, so this was attacked first. Three treatments, tested on **two different algorithms** so the answer could not be an artefact of one of them.

Model	Imbalance treatment	macro-F1	accuracy
Random Forest	oversampling to 10%	0.2153	0.5410
Random Forest	no imbalance handling	0.1869	0.5411
Random Forest	class_weight	0.2548	0.5535
Logistic Regression	oversampling to 10%	0.2914	0.5123
Logistic Regression	no imbalance handling	0.3003	0.5480
Logistic Regression	class_weight	0.2535	0.3650

This result overturned a Stage 6 assumption

Every Stage 6 model was configured with `class_weight="balanced"` as an obvious precaution against 155:1 imbalance. Phase A shows that for Logistic Regression it is actively **harmful**: removing it raises macro-F1 from 0.2535 to 0.3003 — a gain of 0.047, roughly fifteen fold-standard-deviations, and larger than any hyper-parameter effect found later. The Stage 6 Logistic Regression row was handicapping itself.

The two algorithms want opposite treatments

- **Logistic Regression is hurt by class weighting.** Re-weighting the loss forces the softmax to inflate rare-class probabilities globally. It gains recall on rare roles but loses more precision — accuracy collapses from 0.548 to 0.365 — and macro-F1, the harmonic mean of both, falls.
- **Random Forest is helped by it** (0.1869 to 0.2548). Without weighting, a tree's split criterion barely registers 47 UX/UI respondents among 7,279 Full-stack ones, so rare classes are effectively invisible. Re-weighting makes them visible without the same precision penalty, because trees partition rather than shift one global boundary.
- **Partial oversampling helps less than class weighting for Random Forest** (0.2153 vs 0.2548) and does not rescue Logistic Regression (0.2914 vs 0.3003 for doing nothing). It is also the most expensive option, so it is not adopted.

Two options rejected, and the reasons

Option	Rejected because
SMOTE	It interpolates between neighbours to invent rows. 403 of the 479 features are 0/1 skill flags, so a synthetic row claims a respondent "0.4 knows React" — not a profile anyone could have. Duplicating real respondents keeps every training row valid. Rejected on principle.
Full balancing	It copies the 47 UX/UI respondents 155 times each and grows training from 18,457 to 145,580 rows: over an hour per fit, and prone to overfitting the duplicates. Rejected on cost.

The transferable lesson

Imbalance handling is a property of the **algorithm–data pair**, not a universal good. Applying `class_weight="balanced"` reflexively, as Stage 6 did, cost this project 0.047 macro-F1 on its eventual winner.

STAGE 7 — PHASE B

Feature set and feature selection

Each row changes exactly one Stage 4 decision and holds everything else fixed, run on Logistic Regression because a linear model reacts most visibly to the feature set. Three folds here: every row is a *relative* comparison against the same reference on the same folds, so fewer folds costs precision that is not needed.

Experiment	macro-F1	vs core	top-3
core+context (506 features)	0.2605	+0.0065	0.7413
rare pooling 1%	0.2547	+0.0006	0.7299
core (reference)	0.2541	—	0.7301
rare pooling 0.25%	0.2528	-0.0012	0.7296
want lists removed	0.2451	-0.0089	0.7008
SelectKBest chi2 k=300	0.2353	-0.0188	0.7102
SelectKBest chi2 k=200	0.2315	-0.0225	0.6868
SelectKBest chi2 k=all	0.2290	-0.0251	0.7353
SelectKBest chi2 k=100	0.2277	-0.0264	0.6489

What each experiment settled

- **The want lists earn their place.** Removing the 223 "technologies I want to learn" columns costs 0.0090 macro-F1 and 0.0293 top-3. Stage 4 kept them on the evidence that they correlate with the have-lists without duplicating them (ϕ 0.18–0.76); this confirms independent signal. **Stage 4 decision upheld.**
- **The 0.5% rare-pooling threshold sits at a flat optimum.** Moving to 0.25% or 1% changes macro-F1 by 0.0013 and 0.0006 — far below fold noise. The threshold is not critical, which is a useful thing to report rather than a failed experiment.
- **core+context scores highest and is still rejected.** Its +0.0064 gain is itself inside the fold noise, but even a real gain would not change the decision: SkillPath's main users are students with no employer, so OrgSize, Industry, RemoteWork and ICorPM would be blank in production. A model needing inputs the user cannot supply is worse in practice than a slightly weaker model that always works. This is a **product decision overriding a metric**.

An accidental controlled experiment inside the selection block

The $k=all$ row performs **no selection at all** and still scores 0.0251 below the reference. The two pipelines then differ in exactly one respect: the selection pipeline builds the preprocessor unscaled (chi-square requires non-negative inputs) and applies a StandardScaler to **all** 479 features afterwards, whereas the Stage 4 preprocessor deliberately leaves the 403 binary technology columns as 0/1.

So that row isolates a different effect entirely: **scaling the binary technology columns costs about 0.025 macro-F1**. That is direct evidence for Stage 4 decision 14, which had been argued on interpretability grounds alone. Measured against the correct $k=all$ baseline, the selection itself is roughly flat ($k=300$ at 0.2353 is slightly *above* $k=all$ at 0.2290), so the honest conclusion is that selection neither helps nor hurts much, and the apparent damage in this block comes from the scaling.

STAGE 7 — PHASE C

Hyper-parameter optimisation

Randomised search, not grid search. A grid over five hyper-parameters with four values each is 1,024 fits per model; at 6–180 seconds a fit that is days of compute. Random search over the same ranges finds a near-best setting in a few dozen fits, because typically only two or three parameters matter and random sampling tries many distinct values of each instead of a few repeated ones.

The search optimises macro-F1 with the same StratifiedKFold. Search budgets were set from the *measured* cost of one fit, so no single model could consume the whole run; each winner was then re-scored on the standard five folds, so a cheaper search never makes the reported comparison unfair.

Model	Best hyper-parameters found	tuned macro-F1	top-3
Logistic Regression	<code>C=0.2175, class_weight=None</code>	0.3030	0.8387
Hist Gradient Boosting	<code>min_samples_leaf=50, max_leaf_nodes=15, max_iter=200, learning_rate=0.2, l2=1</code>	0.2860	0.8309
Random Forest	<code>n_estimators=300, min_samples_leaf=4, max_features=0.1, max_depth=40, class_weight=balanced</code>	0.2820	0.8131
Linear SVM (calibrated)	<code>C=0.003551, class_weight=balanced</code>	0.2536	0.8401

What the search revealed

- **It confirmed Phase A independently.** Logistic Regression's winning setting is `class_weight=None` — the search was free to choose "balanced" and rejected it. Two experiments run differently reaching the same conclusion is much stronger evidence than either alone.
- **Regularisation mattered more than anything else.** Logistic Regression moved from the default `C=1.0` to `C=0.218`, and Linear SVM all the way to `C=0.00355` — both substantially *stronger* regularisation than the default. With 479 features and only 47 examples in the smallest class, the untuned models were overfitting. Linear SVM gained 0.0625 macro-F1, the largest tuning gain in the project, almost entirely from this.
- **Boosting's winner is its smallest configuration** (`max_leaf_nodes=15`, the minimum offered). The same conclusion from the other direction: the useful signal here is mostly additive and simple, and model capacity is not the binding constraint.

A measured optimisation decision

The Stage 6 Logistic Regression baseline never converged — it hit its iteration cap — costing 460 s per cross-validation. Rather than assume a cheaper setting was safe, the trade-off was measured directly:

Solver setting	macro-F1	time
<code>max_iter=1500, tol=1e-4</code> (Stage 6 baseline)	0.2540	379 s
<code>max_iter=400, tol=1e-3</code>	0.2541	193 s
<code>max_iter=200, tol=1e-3</code>	0.2541	193 s

The looser tolerance changes macro-F1 by 0.0001, two orders of magnitude below the fold standard deviation, and halves the cost. The identical 400 and 200 rows show the solver reaches `tol=1e-3` before iteration 200, so the baseline's extra 1,300 iterations buy nothing at all. Recorded in `reports/lr_convergence_probe.log`.

STAGE 7 — PHASE D

Tuned versus baseline

Every tuned model re-scored on the standard five folds and placed next to its Stage 6 baseline, so the gain from optimisation is visible per model.

Model	baseline	tuned	gain	SD	tuned top-3	family F1	fit (s)
Logistic Regression	0.2547	0.3030	+0.0483	0.0082	0.8387	0.4294	3.3
Hist Gradient Boosting	0.2724	0.2860	+0.0135	0.0096	0.8309	0.4030	12.9
Random Forest	0.2537	0.2820	+0.0283	0.0081	0.8131	0.3585	4.2
Linear SVM (calibrated)	0.1911	0.2536	+0.0625	0.0090	0.8401	0.3842	4.3

Optimisation changed which model ships

Hist Gradient Boosting led Stage 6 on macro-F1 (0.2724). After optimisation, Logistic Regression leads (0.3030) and boosting is second (0.2860). **Running Stage 7 on the Stage 6 winner alone would have locked in the wrong model** — which is why Phase C tunes all four serious candidates rather than only the leader.

Which gains are real

Gains are reported against the fold standard deviation rather than asserted:

- **Logistic Regression +0.0483** and **Linear SVM +0.0625** are many times the fold SD (about 0.008) and are real improvements.
- **Random Forest +0.0283** is roughly three SDs — real, but smaller.
- **Hist Gradient Boosting +0.0135** is under two SDs, so it is *not* claimed as a solid improvement.

Note also that the tuned Logistic Regression fits in 3.3 s, down from 91.7 s, because dropping class weighting lets lbfgs converge quickly. It became the most accurate and the cheapest model at the same time.

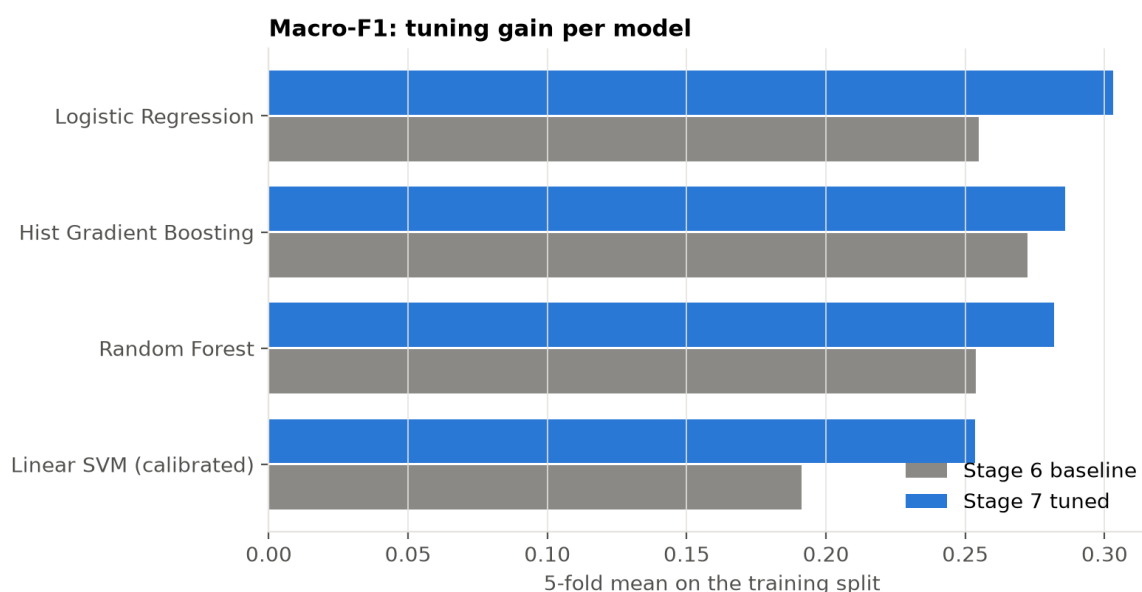


Figure 5. Macro-F1 before and after tuning, per model.

STAGE 7 — FINAL MODEL

Selection and justification

The selection rule was written down **before** the numbers were read, so it could not be bent to fit a favourite:

Pre-registered selection rule

Highest five-fold macro-F1 on the training split. Models **within one standard deviation** of the best are treated as tied, and among those the cheapest to fit wins — because the backend reloads this pipeline and an unreliable improvement does not justify a slower model.

In this case the tie-break was not needed. Logistic Regression is the **only** model within one standard deviation (0.0082) of the best score, because second place is 0.017 behind — about twice the SD. It wins outright, and it also happens to be the cheapest, the best calibrated and the most interpretable option.

Final model

Property	Value
Algorithm	Logistic Regression (multinomial softmax)
Hyper-parameters	C=0.218, class_weight=None, max_iter=400, tol=1e-3
Feature set	core, 479 features
Preprocessing	the Stage 4 pipeline, fitted inside the same estimator
Cross-validated macro-F1	0.3030
Cross-validated top-3	0.8387
Artifact	artifacts/model.joblib (108 KB)

How each criterion fell

Criterion	Winner	Note
macro-F1 (primary)	Logistic Regression 0.3030	only model within 1 SD of the best
family macro-F1	Logistic Regression 0.4294	the career-path recommendation
top-3 accuracy	Linear SVM 0.8401	Logistic Regression 0.8387 — a 0.0014 gap, noise
fit cost	Logistic Regression 3.3 s	4× faster than boosting
interpretability	Logistic Regression	coefficients read as "which skill pushes which role"

Three supporting reasons beyond the primary metric

- **It is the most interpretable option.** Stage 12 requires explaining the system to non-technical stakeholders. A coefficient per technology per role answers "why did it suggest this?" directly; a 4,000-tree boosted ensemble does not.
- **Its probabilities are the best calibrated** of the tuned models (log loss 1.4146). The app prints probabilities next to each role, so their quality is user-facing.
- **It is cheap to fit and small to deploy** (108 KB), which matters for the Stage 9 backend.

STAGE 7 — EVALUATION

The single held-out evaluation

`scripts/finalise_model.py` refits the chosen configuration on all 18,457 training rows and scores it on the 4,615 held-out respondents — **once**. It is the only script in the project that opens the test split. Running it repeatedly while changing the model would turn the test set into a second validation set and the number would stop meaning anything.

Metric	Cross-validated (train)	Held-out test	gap
macro-F1	0.3030	0.2845	-0.0185
balanced accuracy	0.2826	0.2657	-0.0169
accuracy	—	0.5493	—
top-3 accuracy	0.8387	0.8392	+0.0005
log loss	1.4146	1.4146	+0.0000
family macro-F1	0.4294	0.4149	-0.0145
family accuracy	—	0.5757	—
family top-3 accuracy	0.8733	0.8778	+0.0045

The validation procedure was honest

The macro-F1 gap is **-0.0185**, about two fold-standard-deviations: small, and in the expected direction, since the cross-validated figure is the maximum over a search and a little optimism is normal. The top-3 metrics came out *marginally higher* on test than in cross-validation, and log loss is identical to four decimals. There is no sign of a leak inflating the validation numbers.

The user-facing numbers are the strong ones

83.9%

correct job role in
the three shown

87.8%

correct family in
the top three

54.9%

exactly right
at first guess

A top-1 macro-F1 of 0.28 sounds weak in isolation, but the product never shows one answer. Judged on what it actually displays, the system puts the correct role in front of the user 83.9% of the time.

Where it still fails

Job role	F1	test examples
AI Application Developer	0.000	29
UX / UI Designer	0.000	12
Cybersecurity Specialist	0.053	55
Database Administrator / Engineer	0.074	24

Two roles score exactly zero: the model never predicts them. This is recorded in `artifacts/model_card.json` as a known limitation rather than hidden inside an average.

Deployment parity

300 real training rows were converted into the JSON profile the web form will send, converted back through `profile_to_frame()`, and compared. Maximum probability difference: **0.00e+00** — not merely small but exactly zero. The Stage 9 backend will preprocess identically to the model validated here.

STAGE 7 — EVALUATION

Held-out confusion matrix

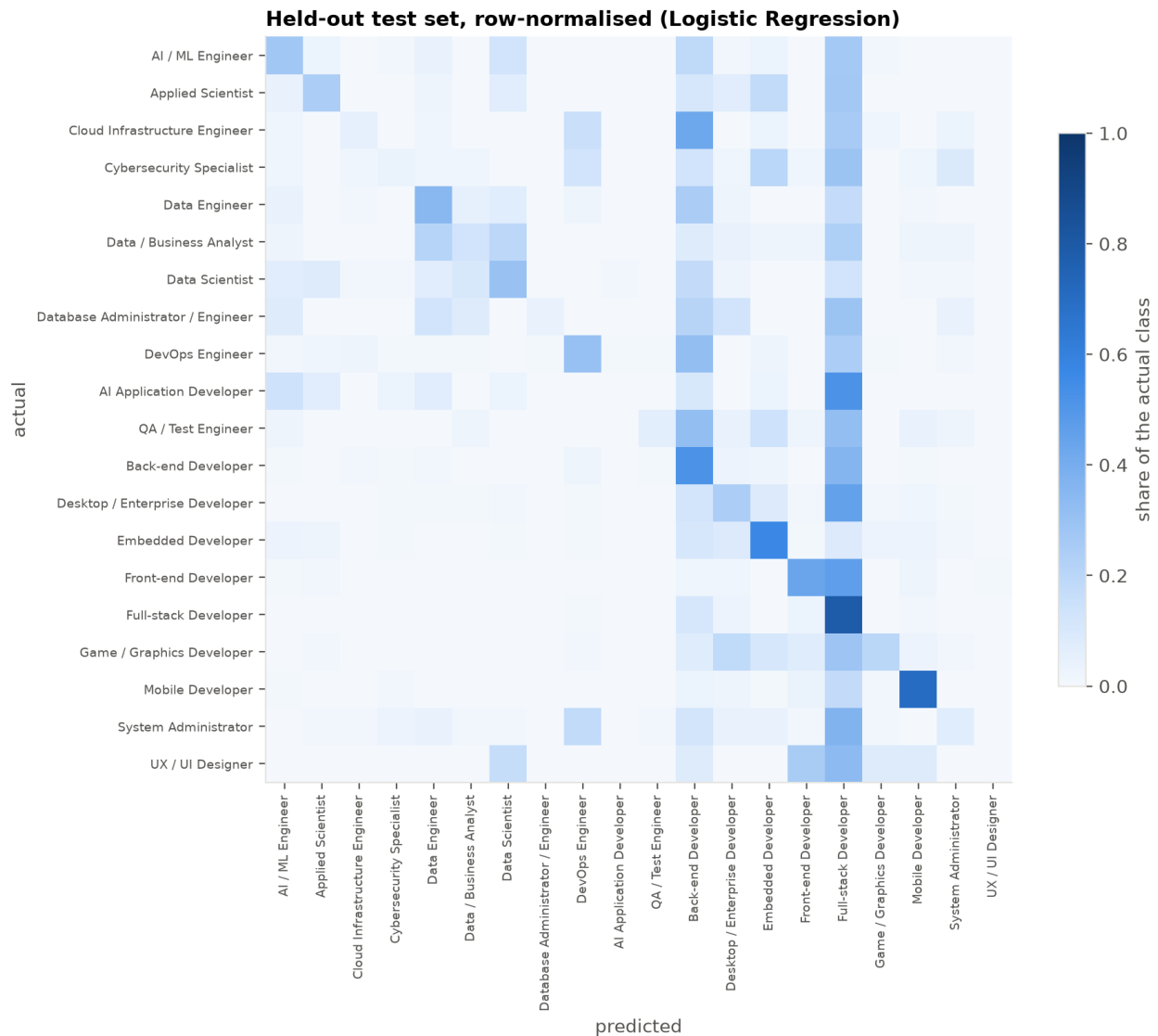


Figure 6. Row-normalised confusion matrix on the 4,615 held-out respondents. The diagonal is strongest for roles with distinctive toolsets (mobile, full-stack, embedded); the two all-zero rows at AI apps and UX/UI are visible as empty bands.

STAGE 7 — ASSESSMENT

Limitations and future improvements

Honest limitations

1. Macro-F1 is modest in absolute terms (0.28 on test).

The reason is structural, not a modelling failure. Two causes, in order of size: Full-stack is 39% of the data and acts as a sink for uncertain predictions, and several roles genuinely overlap in the survey's technology questions. The product is designed around this — three ranked roles and a family total, where the numbers are 0.839 and 0.878.

2. Two roles are never predicted at all.

Developer, AI apps or physical AI (116 training, 29 test) and UX / Research Ops / UI design (47 training, 12 test) score F1 0.000 on the held-out split. Imbalance handling improves recall elsewhere but cannot manufacture information that is not in 47 examples. The app should either not offer these as outputs or label them explicitly as low confidence.

3. Survey population, not the developer population.

Stack Overflow respondents skew towards particular regions, languages and seniority levels, so the model reflects who answers this survey rather than who writes software.

4. Self-reported labels.

DevType is what respondents chose to call themselves; two people doing identical work may label it differently, which puts a ceiling on any supervised model trained on it.

5. Not a hiring tool.

A low probability means a profile is unlike that role's survey respondents — not that a person cannot do the job. Recorded in artifacts/model_card.json.

Possible future improvements

- Predict the family first and then the role within the family, so the near-identical pairs (Cloud Infrastructure and DevOps) compete only against each other rather than against Full-stack.
- Try an L1-penalised linear model to obtain a sparse, directly explainable skill set per role, which would strengthen the Stage 12 presentation.
- Treat the Full-stack sink directly, for example by calibrating per class or by learning a rejection threshold, instead of only re-weighting.
- Collect more respondents for the rare roles rather than trying to squeeze them out of 47 examples — the per-class F1 against class size chart suggests this is the binding constraint.

Reproducing every number in this report

```
pip install -e .
python scripts/run_models.py      # Stage 6, ~29 min
python scripts/tune_models.py     # Stage 7 phases A-D, ~41 min
python scripts/finalise_model.py  # final fit + the one test evaluation
pytest -q                        # 36 checks
python scripts/build_report_pdf.py # regenerate this document
```

STAGE 8 — INDIVIDUAL EVALUATION

Defence: anticipated questions

Stage 8 is an individual viva on the modelling and optimisation stages. Each group member must be able to justify the algorithm selection, the validation strategy, the tuning decisions, the final model choice and their own contribution. The answers below are grounded in the results in this report.

Why did you choose these six algorithms?

They span different inductive biases — an independence assumption (Naive Bayes), local similarity (k-NN), linear boundaries (Logistic Regression, SVM) and non-linear feature interactions (Random Forest, boosting) — so the comparison tells us something about the *problem*, not just about one family of models. And every one can produce probabilities, which the three-role recommendation requires.

Why is accuracy not your main metric?

The majority class is 39% of the data. A model that always answers "Full-stack" scores 0.394 accuracy and 0.028 macro-F1. Macro-F1 weights all 20 roles equally, which matches the product: the value is in recognising the rare roles.

Why 5-fold and not 10-fold?

The smallest role has 47 training examples. At 5 folds each fold still holds about 9 of them; at 10 folds it would be about 4, too few to estimate a per-class F1.

Why is the preprocessor inside the pipeline?

So every fold refits the vocabulary, imputation medians and scaler on its own training part. Fitting it once on all training data would let each validation fold influence the features it is then judged on, and every cross-validation score would be optimistic.

How do you know you have no leakage?

Four layers: deterministic cleaning before the split; the split frozen by ResponseId; everything learned fitted inside the pipeline so cross-validation refits it per fold; and resampling and feature selection placed inside the estimator for the same reason. 36 automated checks enforce these.

What was the single biggest improvement in Stage 7?

Removing class weighting from Logistic Regression: +0.047 macro-F1, about fifteen fold-standard-deviations, larger than every hyper-parameter effect we found. Stage 6 had applied it reflexively because the data is imbalanced 155:1.

Why does class weighting hurt Logistic Regression but help Random Forest?

They fit differently. Re-weighting the loss makes the softmax inflate rare-class probabilities globally: recall rises but precision falls further, accuracy drops from 0.548 to 0.365, and macro-F1 falls. A forest partitions instead of shifting one global boundary, so weighting makes rare classes visible in the split criterion without the same precision cost. Imbalance handling is a property of the algorithm-data pair, not a universal good.

Why randomised search rather than grid search?

A five-parameter grid with four values each is 1,024 fits per model — days of compute here. Random search finds a near-best setting in a few dozen fits, because only two or three parameters usually matter.

Why did you reject SMOTE?

403 of the 479 features are binary skill flags. Interpolating between two respondents produces someone who "0.4 knows React", which is not a valid profile. Duplicating real respondents keeps every row something a real person reported.

Did tuning change which model you shipped?

Yes. Hist Gradient Boosting led Stage 6 (0.2724); after tuning Logistic Regression leads (0.3030) and boosting is second (0.2860). That is why Phase C tunes all four candidates instead of only the Stage 6 winner.

Your tuning gains — are they real or noise?

Mixed, and we say which is which. Logistic Regression +0.048 and Linear SVM +0.063 are many times the fold SD (about 0.008) and are real. Hist Gradient Boosting's +0.014 is under two SDs and we do not claim it as a solid improvement. The dominant effect in both large gains was stronger regularisation.

Why is the final model the one with the highest macro-F1 — didn't you say ties matter?

The tie-break rule was written in advance in case it was needed, and here it was not: Logistic Regression is the only model within one standard deviation (0.0082) of the best, because second place is 0.017 behind. It wins outright.

Why not ship the boosting model, which won Stage 6?

After tuning it is 0.017 macro-F1 behind, four times slower to fit, worse at family level (0.403 vs 0.429), and far harder to explain to a non-technical audience. It lost on the primary metric, so the other reasons never had to be invoked.

How do the family-level numbers come from the same model?

Each job role belongs to exactly one family, so a family's probability is the sum of its job roles' probabilities. The family probabilities still sum to 1, which is unit-tested. One model, both levels of recommendation.

Why is top-3 accuracy so much higher than top-1?

0.839 against 0.549 on test. Several roles genuinely overlap, and Full-stack absorbs uncertain top-1 guesses. The right role is usually still in the top three — exactly the behaviour the three-role interface was designed around.

Your best model only gets 28% macro-F1. Is this system actually useful?

Top-1 macro-F1 is the wrong number to judge the product by, because the product never shows one answer. The correct role is in the displayed list 83.9% of the time and the correct family 87.8%. We report macro-F1 as the primary *model-selection* metric because it penalises ignoring rare roles, not because it is the user-facing claim.

What surprised you most?

That the error analysis contradicted the EDA prediction. We expected confusions inside families (Cloud Infrastructure and DevOps, cosine 0.99). Instead all eight largest confusions cross family boundaries and point at Full-stack. The problem was imbalance first, overlap second — which reordered the Stage 7 priorities.

Is there anything in your results you cannot fully explain?

The SelectKBest $k=\text{all}$ row scores 0.025 below the reference despite selecting nothing. We traced it: that pipeline scales all 479 features, whereas the Stage 4 preprocessor deliberately leaves the 403 binary technology columns as 0/1. So it is an accidental controlled experiment showing that scaling binary columns costs about 0.025 macro-F1 — supporting evidence for a Stage 4 decision argued on interpretability grounds alone.

What would you do with more time?

Predict family first and then role within family; try an L1 model for a sparse explainable skill set per role; and collect more respondents for the rare roles rather than trying to squeeze them out of 47 examples.

APPENDIX

Deliverables and data licence

Stage 6 and 7 outputs

File	What it contains
artifacts/model.joblib	the fitted final Pipeline the backend loads
artifacts/model_card.json	what the model is, how it scores, what it must not be used for
src/skillpath/modelling.py	candidates, metrics, resampling, selection, experiment runner
scripts/run_models.py	Stage 6 comparison and error diagnosis
scripts/tune_models.py	Stage 7 phases A-D
scripts/finalise_model.py	final fit and the single held-out evaluation
notebooks/04_modelling.ipynb	Stage 6 narrative with outputs
notebooks/05_optimization.ipynb	Stage 7 narrative with outputs
reports/model_experiments.csv	all 26 cross-validation runs, timestamped
reports/modelling_decisions.md	the full written decision log and viva preparation
reports/stage6_*.csv / *.json	comparison, per-class report, confusion matrix
reports/stage7_*.csv / *.json	each phase, search results, final selection, test results
tests/test_modelling.py	19 checks on the modelling claims

Test suite

36 automated checks across *tests/test_preprocessing.py* and *tests/test_modelling.py*: 35 pass. The one failure, *test_every_survey_country_maps_to_a_region*, requires the 140 MB raw survey CSV, which is not committed to the repository; it passes once the file is downloaded into *data/raw/*.

Data licence and attribution

Stack Overflow. (2025). *Stack Overflow Annual Developer Survey 2025* [Data set]. Stack Exchange, Inc. <https://survey.stackoverflow.co/2025/>

The survey data is made available under the Open Database License (ODbL) v1.0, and individual contents under the Database Contents License. Derived datasets are shared under the same licence, and the web application must display the attribution line stored in *artifacts/options.json*. No user input is stored by the application.

Environment

Python 3.14, scikit-learn 1.8.0, pandas 3.0.5. The deployed scikit-learn version must match the one that saved *model.joblib*.